# Computer Architecture Lab1 Report

**Ray** **2026-04-25**


1 实验要求🤪 👺 😠 ..... 2
2 单周期 CPU 架构 ..... 2
3 流水线 CPU 架构 ..... 2
3.1 基础架构 ..... 2
3.2 冒险解决机制 ..... 2
3.3 异常处理与精确异常 ..... 2
3.4 模块化项目结构 ..... 3
3.5 多周期乘法器 & 除法器 ..... 3
3.5.1 乘法器 ..... 3
3.5.2 除法器 ..... 4
3.6 问题与解决方案 ..... 4
4 性能测试 ..... 5
4.1 测试环境与参数说明 ..... 6
4.2 性能数据深度分析 ..... 6

# 1 实验要求🤪 😡 😠

本次实验的主要目标是使用 SystemVerilog 设计并实现一个 RISC-V 架构的 CPU。具体要求包括：

1. **基础单周期**：实现支持基础整数指令集的单周期 CPU。
2. **流水线改造**：在单周期基础上，将其改造为经典的五级流水线架构。
3. **冒险处理**：解决流水线中的数据冒险与控制冒险。
4. **验证与测试**：接入 Difftest 测试框架，保证指令提交时的架构状态与模拟器完全一致，并通过 `lab1-test.bin` 等功能测试。

# 2 单周期 CPU 架构

单周期 CPU 是流水线设计的基础。在单周期架构中，每条指令的获取、译码、执行、访存和写回都在一个时钟周期内完成。虽然每指令周期数严格为 1.0，但由于关键路径过长（通常是从指令存储器经过 ALU 再到数据存储器），导致 CPU 的最高时钟频率非常低。

# 3 流水线 CPU 架构

## 3.1 基础架构

本实验实现了一个经典的五级流水线（取指、译码、执行、访存、写回）。此外，为了与 Difftest 框架的严格时序要求对齐，我们在写回阶段后新增了一个完整的提交阶段寄存器。这样可以完美解决时序逻辑（寄存器堆的时钟上升沿写回）与组合逻辑（Difftest 的检查点）之间的单周期同步偏差问题。

我们在各个流水段之间加入了段间寄存器，将指令的数据流与元数据打包传递，践行了“一次译码，全局使用”的设计哲学，极大简化了后级流水线的控制逻辑。

## 3.2 冒险解决机制

**数据冒险与数据转发** 为了解决读后写的数据冒险，我们引入了前递单元。当执行阶段或访存/写回阶段的指令目标寄存器与当前译码阶段指令的源寄存器发生重叠时，直接将旁路网络中的最新数据转发给执行阶段，避免了不必要的流水线停顿，提高了指令吞吐量。

**控制冒险** 当遇到分支指令时，流水线会产生控制冒险。由于我们尚未引入动态分支预测器，目前采用的是静态预测不跳转结合冲刷机制。当执行阶段计算出实际需要跳转时，会将前端段间寄存器中的错误指令冲刷为气泡，并重置程序计数器重新取指。

## 3.3 异常处理与精确异常

为了正确处理程序退出时的自定义指令以及潜在的内存越界或非法指令，我们实现了一套基于“毒药位”的精确异常机制。

**非法指令标记** 当译码单元遇到无法识别的操作码时，不会立即触发停机，而是将其标记为非法指令并沿流水线向下传递。

**幽灵指令过滤** 只有当该带毒指令真正到达提交阶段，且没有被前面的分支指令冲刷掉时，才会触发异常。这完美解决了由于分支预测失败导致前端错误抓取垃圾内存而引起的幽灵指令误报问题。

有了该异常处理机构，整个项目就可以采用构造正确的范式，防止流水线上层的异常影响到后续步骤的进行。

## 3.4 模块化项目结构

本项目在架构设计上采用了高度解耦的模块化思想。顶层模块作为主板，负责实例化各个流水段并处理段间连线。其中执行阶段（EXU）被设计为一个极其灵活的调度封装层：

执行单元内部不再处理复杂的数据冒险，而是通过独立的“操作数解析单元”统一接收寄存器数据和前递网络数据，输出最准确的计算数。随后，执行单元根据指令类型，将操作数分发给底层的算术逻辑单元（ALU）、多周期乘法器或多周期除法器。这种分发与执行分离的架构，使得未来扩展浮点运算单元或自定义加速器变得非常容易。

## 3.5 多周期乘法器 & 除法器

### 3.5.1 乘法器

**原理** 基于分块移位加法原理。由于单周期内完成 64 位乘法会导致关键路径过长，我们将计算过程按参数化周期进行拆分。例如设定为 4 周期时，每周期仅处理 16 位的乘法累加，通过状态机控制移位与累加过程，以此换取更高的主频。

**实现** 模块内部维护了操作数移位寄存器和累加器。在收到使能信号的首个周期捕获操作数，随后在剩余周期内执行 `accumulator + op1 * op2[SEG-1:0]` 操作，并同步对操作数进行逻辑移位。

```
accumulator ≤ accumulator + op1_reg * op2_reg[SEG - 1 : 0];
op1_reg ≤ op1_reg << SEG;
op2_reg ≤ op2_reg >> SEG;
remain_cycles ≤ remain_cycles - 1;

```

计算完成后拉高完成信号，与执行单元进行握手并解除流水线冻结。

```
end else if (ack) begin
  done_reg ≤ '0;
```

```
end else if (remain_cycles ≠ '0) begin
   ...
  if (remain_cycles = 1) done_reg ≤ 1'b1;

```

### 3.5.2 除法器

**原理** 采用恢复余数法构建 64 周期状态机。除法器本质上是处理无符号运算的引擎，因此在输入阶段需要对有符号数进行绝对值转换。状态机通过逐次左移被除数并尝试减去除数，若够减则商位置 1，否则置 0。

**实现** 在首个时钟周期，模块利用组合逻辑将操作数转换为绝对值并存入 128 位移位寄存器中，同时锁存符号信息。

```
dividend    ≤ {64'b0, abs_op1};
divisor     ≤ {1'b0, abs_op2, 63'b0};
 ...
quo_neg_reg ≤ (op2 ≠ 0) && (op1_neg ^ op2_neg);
rem_neg_reg ≤ op1_neg;

```

在随后的 64 个周期中进行移位减法迭代。最终利用首个周期锁存的符号信息，通过后处理组合逻辑对商和余数进行符号还原。

```
end else if (counter ≠ '0) begin
  if (dividend ≥ divisor) begin
    dividend ≤ dividend - divisor;
    accumulator ≤ (accumulator << 1) | 64'b1;
  end else begin
    accumulator ≤ accumulator << 1;
  end
  divisor ≤ divisor >> 1;
  counter ≤ counter - 1;

```

## 3.6 问题与解决方案

| 核心问题 | 场景与表现 | 架构解决方案 |
|---|---|---|
| 数据转发优先级 | 访存阶段和写回阶段同时尝试向执行阶段的同一源 | 在 SV 组合逻辑块中，先判定写回阶段（较老）前递条件，后判定访存阶段（较新）前递条件，通 |

| | | |
|---|---|---|
| | 寄存器前递数据时，较老数据可能覆盖最新数据。 | 过**物理代码赋值的先后顺序**保证最新数据生效。 |
| 瞬态数据陷阱(数据失效) | 长周期乘除运算冻结了前端，但后级流水线排空导致前递网络提供的数据在第 2 周期彻底消失。 | 在乘除法器内部引入**元数据锁存机制**。第 0 周期利用 D 触发器永久锁存传入的操作数和符号位状态，彻底隔离外部流水线波动。 |
| 除法器运算数截位 | M 扩展的 32 位字指令传入 64 位物理寄存器，若高 32 位存在垃圾数据，将导致状态机得出荒谬结果。 | 在 ALU 内部构建**动态预处理通道**。识别字操作标志，动态对 32 位操作数进行零扩展或符号扩展至 64 位，保证状态机运算正确。 |
| 除数=0 非法情况 | 除数为 0 时商应当为全一（-1）。单纯按数学逻辑将产生的 −1 取补码翻转，会错误变为 +1。 | 利用恢复余数法除零自然生成全 1 的特性，在首周期锁存负号标志时加入屏蔽条件：**仅当除数非零且输入操作数异号时**，才允许取补码。零硬件成本符合规范。 |

# 4 性能测试

我们将单周期与流水线 CPU 的性能进行了对比。评估 CPU 性能的核心公式为：

$$\text{Execution Time} = \text{Instruction Count} \times \text{CPI} \times \left(\frac{1}{F_{\max}}\right)$$

| Architecture | Difftest IPC (lab1/ lab1-extra) | Critical Path Delay (ns) | Estimated Frequency (MHz) |
|---|---|---|---|
| Single Cycle | 0.500 / 0.500 | 231.404 | 4.3 |
| Pipeline (Mul Cycle=2) | 0.500 / 0.038 | 15.849 | 63.1 |
| Pipeline (Mul Cycle=4) | 0.500 / 0.038 | 13.686 | 73.1 |
| Pipeline (Mul Cycle=8) | 0.500 / 0.038 | 13.688 | 73.1 |
| Pipeline (Mul Cycle=16) | 0.500 / 0.037 | 13.689 | 73.1 |
| Pipeline (Mul Cycle=32) | 0.500 / 0.035 | 11.044 | 90.5 |

## 4.1 测试环境与参数说明

本测试基于逻辑综合工具（如 Vivado）生成的时序报告来提取关键路径延迟（Critical Path Delay），并借此估算物理硬件所能达到的最高主频（$F_{\max}$）。 在指令吞吐率（IPC）的测试中，使用了两个不同的测试集：

- `lab1`：以基础的 RV64I 整数运算与控制流为主。
- `lab1-extra`：高强度包含 RV64M 扩展的乘法与除法指令，用于压榨多周期算件的性能。

测试中的自变量 `Mul Cycle` 代表乘法器状态机被参数化拆分的周期数。周期数越小，单周期内需要计算的乘法位宽越大。

## 4.2 性能数据深度分析

数据清晰地揭示了计算机体系结构在物理实现层面的四大客观规律：

1. **流水线带来的频率飞跃**：单周期架构由于必须在一个时钟周期内完成 64 位除法的全部组合逻辑（实质上是将 64 个减法器在物理空间上级联），导致关键路径延迟高达 231.4 ns，预估最高频率仅为 4.3 MHz。引入流水线并将乘除法改造为多周期状态机后，延迟瞬间骤降至 15.8 ns，使得芯片的物理主频实现了近 15 倍的飞跃。
2. **DSP 硬宏与布线延迟墙**：在 `Mul Cycle` 为 4、8、16 时，关键路径延迟不仅没有明显下降，反而奇迹般地“平缓”停留在 13.68 ns。这并非逻辑错误，而是触碰到了 FPGA 的物理底层逻辑。综合工具会自动调用芯片内部固化的 DSP 切片（DSP Slices）来执行乘法。无论运算宽度如何，进入同一物理 DSP 模块的内部延迟与外部在芯片上的布线通勤延迟（Net Delay）是固定的，从而形成了一道物理延迟墙。
3. **进位链的物理极限**：当 `Mul Cycle` 增加到 32 时，单周期乘法位宽缩小至仅 2 bit。此时综合工具放弃了庞大的全局 DSP 路由，转而使用更轻量的纯逻辑资源，使得延迟成功下探至 11.04 ns（约 90.5 MHz）。然而，它无法进一步下降，因为计算最终依赖于 64 位加法器（Accumulator），其进位链（Carry-Chain）的物理波纹传递时间构成了新的性能地板。
4. **IPC 与频率的权衡 (Trade-off)**：对于 `lab1-extra` 这种乘除法密集型测试，随着 `Mul Cycle` 的增加（从 2 增加到 32），由于执行单元冻结流水线的时间变长，每周期指令提交数（IPC）出现了可见的下滑（从 0.038 降至 0.035）。这完美印证了架构设计中的核心矛盾：用牺牲指令级并行度（IPC）的代价，换取更短的关键路径与更高的全局时钟频率（$F_{\max}$）。